

SE LAB 3

Raghav G Uddihal

PES1UG24AM215

AIML D

Written Justification:

Architecture Selection: I chose **Microservices Architecture** for the Hyperlocal Courier Dispatch & Tracking Engine System.

Architectural Choice & Reasons:

Asynchronous Workload Isolation: The system handles two radically different traffic profiles: sporadic, high-consistency order placement and continuous, high-frequency real-time GPS telemetry streaming from active riders. A microservices approach decouples the *Tracking & Telemetry Component* from the *Order Management Component*, ensuring that high-volume location pings do not starve core dispatch operations of computing resources.

Independent Component Scalability: Multi-stop route calculations and geospatial radius matching (3 km matching threshold) are computationally expensive. Microservices allow the *Matching & Dispatch Component* and *Route Optimization Component* to scale horizontally independent of static modules during peak delivery windows.

Security Advantage: I isolated the *Authentication & Verification Component* within the architecture, enforcing strict token-based boundaries for API requests and separating critical cryptographic workflows—such as destination OTP verification—from public-facing telemetry channels. This ensures that a compromise in the real-time tracking websocket layer does not expose core database credentials or user authorization tokens.

Performance Benefit: By distributing services across lightweight, containerized endpoints communicating over optimized internal networks, I minimized processing bottlenecks. This direct service-to-service communication structure

directly supports the strict non-functional requirement of maintaining live GPS telemetry updates to sender clients with under 2-second latency.

Drawbacks: Despite its benefits, I acknowledge that adopting a microservices architecture introduces notable drawbacks, primarily operational complexity and increased overhead when deploying and monitoring multiple independent services. Furthermore, distributed network communication between components like the matching engine and route optimizer can introduce latency, which presents a challenge when trying to maintain strict real-time telemetry updates

UML Diagram:



